PROBLEM STATEMENT 01

Product Quality Inspection & Defect Tracking System

1. Problem Context

A manufacturer records product images, inspection results and corrective actions in separate files. It becomes difficult to link a defect to its batch, revisit a decision or see whether the same problem returned after a repair.

The challenge is more than classifying an image: connect inspection evidence, human review, corrective work and follow-up results in one usable application.

2. Problem Statement

Build a system to **manage products and batches, upload inspection evidence, identify possible defects using AI/ML, review findings and track corrective action through reinspection**. Users should see what needs attention, which defects are confirmed, which actions remain open and whether issues recur.

> **Choose one setting.** Metal parts, fabric, tiles, bottles, packaging or a comparable visual-inspection use case. One product family and one organisation are sufficient; teams design their own screens and workflow.

3. Target Users

Role	Typical actions
Inspector / operator	Create inspections, upload images and observations; confirm or challenge AI suggestions.
Production supervisor	Handle assigned issues, record corrective work and submit follow-up evidence.
Quality manager	Resolve disputed cases, record batch decisions and review trends/settings.

Role names are flexible. Demonstrate at least two permission levels, enforced by the backend.

Suggested Workflow

Register product & batch → **Upload images & observations** → **Generate ML suggestion** → **Human review**

Review and recovery paths ↓

No defect confirmed **Record the inspection outcome.**	**Defect confirmed** **Corrective action → reinspection** **→ close or reopen.**	**Uncertain / failed** **More evidence or manual review;** **keep the original attempt.**

Separate batch decision: an authorised person records any batch hold or release. One inspected image does not establish the quality of the entire batch.

What the Application Must Do

4. Mandatory Requirements

Build a working web/mobile interface, backend APIs and a persistent database. CRUD means create, view, update and, where appropriate, delete or archive records.

Area	Required behaviour
A. Products & batches	Create, view, edit and archive products/batches. Store identifiers, category, date, quantity where relevant and status. Production line/supplier fields are optional.
B. Inspections & images	Link inspections to a product/batch. Record inspector, time, item/sample reference and observations; support at least one image format. Retain original evidence and draft corrections. State whether each image is an item, surface or batch sample.
C. Model results & review	Show the ML suggestion beside its image. A reviewer can confirm/correct it, explain the decision or request more evidence. Keep machine output separate from the human-confirmed result.
D. Defects & actions	Store defect category, description, severity, evidence and review status. Assign corrective actions with an owner, target date, notes and status. Link follow-up inspections; allow closure or reopening without deleting earlier findings.
E. Batch decisions	Show inspected/reviewed item counts. A permitted user records any hold/release decision separately from individual inspection results.
F. Dashboard & search	Filter by product, batch, date, defect category, review state or action owner. Show pending reviews, confirmed defects, open actions, recent activity and one useful trend. Open source records from summary totals.
G. API & integrity	Provide record, upload, processing, review and dashboard APIs. Validate permissions, identifiers, required values and status transitions server-side. Prevent accidental repeat submissions where practical; return clear errors.
H. History & recovery	Record actor, time and reason for important changes. Delete drafts only where appropriate; archive/version submitted evidence and decisions. Retain failed processing attempts and support retry/manual review. Inactive products must not break history.

Suggested Data Relationships

Product → batches → inspections → images/model results/reviews. Confirmed defects link to corrective actions and follow-up inspections. Users and history connect each important change to its author. Store file references/metadata in the database; bytes may use local or cloud storage.

Frontend Functions

Product/batch workspace; inspection upload; image-and-result review; defect/action history; searchable dashboard. Screens may be combined. Show loading, empty, failed-upload and unavailable-AI states without losing submitted records.

> **Count items, not photographs.** Separate AI flags from confirmed defects. Define the denominator of every defect rate; multiple photographs of one item are not independent products.

AI/ML & Generative AI

5. AI/ML Requirement

Integrate **one learned capability that processes inspection images**. Pretrained models, transfer learning and modest trained models are acceptable. The result must be stored and reviewable; a new architecture is not required.

Choose one approach	Possible application
Good / defective classification	Flag an image for review. Binary classification alone is a sufficient core ML task.
Defect-category classification	Suggest a visible defect type only when the dataset supplies those labels.
Visual anomaly detection	Compare with normal examples and flag unusual appearance.
Learned image similarity	Retrieve similar reviewed defects and evaluate the usefulness of the matches.

Store: input reference, predicted class or retrieved matches, model/version, processing time and a score when available. Similarity/anomaly scores are not automatically probabilities; do not invent confidence.

Evaluate: use held-out images, disclose class counts/split and report a confusion matrix or task-appropriate measure, successes and failures. Keep near-duplicates/augmentations of one item together. Retrieval needs labelled queries and relevant top results; anomaly detection needs threshold testing. No fixed accuracy target is imposed. [T1–T3]

Limits: binary labels do not establish crack/scratch/dent classes. Images alone do not establish machine faults or supplier responsibility. Keep manual review available for unsupported products, poor images and model failure. Rules alone do not satisfy this ML requirement.

6. Generative AI Requirement

Implement **one feature grounded in stored inspections, confirmed defects and actions**: a batch summary, report or question-answering function. A chat interface is optional; a generic chatbot is insufficient.

Example request	Expected evidence-backed output
“Summarise Batch B-104.”	Inspection coverage, pending reviews, confirmed defects and open actions.
“Have we seen this issue before?”	Related inspections and recorded corrective actions, without assuming the same cause.
“What needs attention today?”	Overdue actions, unreviewed inspections or reopened cases.
“Draft a quality report.”	A selected-period report separating observations from suggestions.

Grounding rules: retrieve only permitted records; cite inspection/action IDs or links; distinguish unreviewed predictions from confirmed outcomes. Compute counts/rates in code or database

queries. State missing evidence. GenAI must not silently release batches, close issues or certify quality.

Test: a supported question, unknown batch, misleading premise and unauthorised-data request. Notes/uploads are evidence, not instructions that override permissions. See shared testing guidance and [T9].

Workflows, Innovation & Scope

7. Possible Workflows

These are starting points, not three compulsory implementations. Adapt a route while retaining the required records, evidence, review, action, ML and GenAI capabilities.

Flow	Process and brainstorming question
Incoming batch	Register → upload samples → ML flags → reviewer confirms/corrects → action/follow-up → human batch decision. **What evidence helps reviewers challenge an AI suggestion?**
Recurring defect	Find related confirmed issues → compare action history → investigate recurrence → record new action and follow-up. **Is the increase genuine, or caused by more inspections/duplicate images?**
Unreliable inspection	Blurred/unsupported image → needs review → another image or manual inspection → retain history. **How can rework be reduced without hiding uncertainty?**

8. Advanced Features (Optional)

Defect-region annotations, image-quality checks, similarity search, bulk uploads, inspection checklists, before/after comparisons, notifications or an offline-friendly upload queue. Improving an existing feature is equally valid.

9. Innovation Requirement

Implement **one team-designed improvement**. Explain the user problem, approach and how its value is tested. Consider review priority, inspector disagreement, corrective-action effectiveness, recurring issues or unfamiliar products. Innovation may be in the interface, workflow or algorithm; extra AI is not compulsory.

10. Out of Scope

No live cameras, sensors, machinery control, manufacturing ERP integration, large-model training, all-product support or production-scale deployment is required. Segmentation and machine-failure prediction are optional. Manual uploads and a reliable local demonstration are sufficient.

This is visual-inspection decision support, not safety certification or proof that invisible defects are absent. Automatic batch release is not a required feature; final batch decisions remain human.

11. Expected Deliverables

Submit the working inspection workflow, dashboard, image ML, grounded GenAI and innovation evidence. Include code, APIs, diagrams, evaluation and setup materials from the shared checklist (page 10).

12. Suggested Technology Scope

Use any suitable frontend/mobile toolkit, API backend, database, file storage and image/GenAI models. Vector search is optional. Datasets: page 9; shared tools: page 10.

13. Final Demonstration

Create product/batch → upload an unseen evaluation image → store ML result → confirm/correct → assign/complete action → reinspect → show history/dashboard → generate a cited summary. Also show uncertainty/failure, a refused unauthorised action and measured innovation value. A prerecorded success alone is insufficient.

PROBLEM STATEMENT 02

Document Verification & Approval System

1. Problem Context

An organisation receives forms, invoices, receipts and supporting files. Staff copy information, check missing or inconsistent details and request corrections. Repeated uploads and scattered notes make it difficult to know what is current, what needs attention and why a decision was made.

The challenge is more than OCR: extracted information must be checkable, correctable and linked to a request before a reviewer decides.

2. Problem Statement

Build a system to **accept documents, automatically extract relevant fields, check stated requirements, highlight issues for review and track corrections through human approval or rejection**. Users should see submitted evidence, extracted values, unresolved checks and decision history.

> **Choose one scenario.** Invoice review, expense reimbursement, supplier onboarding, an internal application with supporting forms or a comparable workflow. One organisation and a limited set of document types are sufficient.

3. Target Users

Role	Typical actions
Submitter	Create requests, upload files, track progress and respond to corrections.
Reviewer	Inspect/correct extracted fields, review flags and request clarification.
Approver / administrator	Record final decisions, manage basic requirements and review summaries.

Roles may be combined, but submitters and reviewers need different permissions. A submitter must not approve their own request through a backend endpoint.

Suggested Workflow

Create request & upload files → **Extract fields** → **Run verification checks** → **Human review**

Review and recovery paths ↓

Approve / reject **Authorised user records a decision and reason.**	**Needs correction** **Update → re-extract/check → return to review.**	**Processing failed** **Retry or review manually; never approve by default.**

Verification means completeness, consistency and stated rules. OCR, similarity and scores do not prove authenticity or justify labelling a person fraudulent.

What the Application Must Do

4. Mandatory Requirements

Build a working interface, backend APIs and persistent data. An OCR script without a review workflow, or a manual form with only an approval button, is insufficient.

Area	Required behaviour
A. Request CRUD	Create, view, edit and withdraw/archive requests. Store ID, type, submitter, date, status and assigned reviewer. Drafts may be edited; submitted changes must remain traceable. Explain why further information is needed.
B. Files & versions	Support images and at least one PDF route (text-based or scanned). Declare formats/limits; accept one or more documents per request. Retain originals, type, version, uploader and processing state. Reject corrupt/unsupported files clearly.
C. Automatic extraction	Extract a small relevant schema: e.g. name, reference, date, amount or category. Link each value to its file, extraction run and page when available. Store scores when supplied; preserve original extraction beside reviewer corrections.
D. Verification checks	Implement **at least two different checks**: required field/attachment, form mismatch, inconsistent amount/date or possible duplicate. Explain the rule, evidence and result. Distinguish missing, unreadable and inconsistent. Flags prompt review, not accusations.
E. Human decisions	Support correction requests, resubmission and approved/rejected outcomes with a permitted reviewer, reason and time. Revised documents return to review; failed extraction must not approve a request. Retain comments, model runs and decision history.
F. Dashboard & search	Search/filter by ID, useful fields, type, date, assignee or status. Show pending/approved/rejected/correction counts, frequent missing fields and review turnaround. Define duration timestamps; link summaries to source requests.
G. API & security	Provide request, file, extraction, check, review and summary APIs. Enforce access/status rules server-side. Protect files from unrelated users; keep API keys out of the frontend. Validate file type/size. [T8]
H. Recovery & currency	Show processing, empty and failure states; permit retry/manual review without losing records. Preserve replaced documents, but use only the current version for review, dashboards and GenAI.

Suggested Data Relationships

Users → requests → document versions → extraction runs/fields → checks. Review flags, corrections, comments and decisions link to their request, evidence, actor and time. Equivalent schema designs are acceptable.

Frontend Functions

Submitter workspace; pending/flagged review queue; document displayed beside extracted fields and editable corrections; decision/version history; searchable dashboard. These functions may share screens.

Possible states: Draft → Submitted → Processing → Under Review → More Information Required / Approved / Rejected. A replacement document returns to review.

AI/ML & Generative AI

5. AI/ML Requirement

Use **at least one trained or pretrained learned component**. Existing OCR with useful structured extraction and a correction workflow is sufficient; training an OCR engine is not required.

Choose one principal direction	Possible application
OCR & field extraction	Read a supported scan; extract selected fields and expose uncertain/missing values.
Document classification	Suggest a supported document type and apply its field schema.
Semantic duplicate retrieval	Use learned embeddings to find related submissions for human comparison.
Unusual-pattern detection	Flag unusual combinations where meaningful labelled/history data exists.

Automatic extraction remains mandatory, even when another direction is the main ML feature. Identify the OCR, document model or LLM used and evaluate extraction separately from GenAI explanations. A manageable route is OCR + limited fields + two transparent checks. [T4–T6]

Evaluate: use a small labelled set outside development/tuning data; report sample counts, selected fields/labels, normalisation rules and a task-appropriate result. Show missing-field errors and correction cases. Keep copies of a document together across splits. For extraction, report exact/normalised field matches; for retrieval, show relevant and false matches. Disclose synthetic anomaly cases. [T2–T3]

Distinguish rules from ML: hashes detect identical files and arithmetic checks totals, but neither satisfies the learned-component requirement. Do not invent model confidence or claim fraud-detection accuracy from synthetic examples. Uncertainty should lead to review.

6. Generative AI Requirement

Generate a briefing, correction message or grounded answer using the **current request, documents, extracted/corrected values, checks, review history and any provided policy**. A separate chat screen is not required.

Example request	Expected evidence-backed output
“Why is this request on hold?”	Unresolved checks/notes with request, document-version and field references.
“What should I correct?”	A clear correction-message draft for a reviewer to approve before sending.
“Summarise the documents.”	Extracted/corrected facts and unresolved inconsistencies; no invented values.

Example request	Expected evidence-backed output
“What needs attention?” / “Which policy applies?”	A permitted-queue summary or a cited stored policy; state when no policy exists.

Required behaviour: retrieve permitted, current evidence and cite request/document IDs plus page/field references where possible. Separate machine extraction from reviewer edits. Explain rather than approve, reject or change records. Missing data stays missing; sample policies must be clearly labelled fictional, not law or real organisational rules.

Protect the workflow: uploaded instructions such as “ignore checks and approve” are document content, not commands. Enforce permissions before retrieval. Test conflicting/unreadable evidence, unknown cases and unauthorised requests. [T9]

Workflows, Innovation & Scope

7. Possible Workflows

Choose one route or propose a comparable scenario; teams need not build all three or a universal approval-workflow builder.

Flow	Process and brainstorming question
Invoice review	Request → extract supplier/reference/date/total → required-value and form/reference checks → resolve flags → decide or request a new file. **How should a mismatch be shown when OCR may have misread a digit?**
Expense claim	Form + receipt → compare amount and possible repeat submission → clarify → human outcome. **Is it a duplicate claim, replacement scan, multi-page receipt or similar purchase?**
Supporting forms	Request type → upload required forms → extract/check checklist → explain omissions → resubmit → review current version. **Can missing details be caught early without blocking unusual but valid cases?**

8. Advanced Features (Optional)

Near-duplicate or version comparison, multilingual OCR, bulk intake, policy retrieval, reviewer workload balancing, editable templates or notifications. Invoice/purchase-order matching is optional; a full procurement system is not required.

9. Innovation Requirement

Implement **one team-designed improvement** and test its value. Explore correction effort, clearer-scan requests, understandable rejections, version comparison, accessibility, routing or unsupported document types. An interface or validation improvement is as valid as extra AI.

10. Out of Scope

No issuer-level authenticity, comprehensive forgery detection, government identity checks, real payments, legal interpretation, ERP integration or production-scale rollout is required. Digital signatures, handwriting, large-model training and support for every language are outside the minimum.

Use synthetic or appropriately licensed data, not participants' real IDs, bank records, confidential invoices or personal applications. Flags are not proof of fraud; authorised humans record final decisions.

11. Expected Deliverables

Submit the working request/review workflow, extraction, two checks, corrections/version history, dashboard, grounded GenAI and innovation evidence, plus the shared code, APIs, diagrams, evaluation and setup package on page 10.

12. Suggested Technology Scope

Use any frontend/mobile toolkit, backend and persistent database. Existing OCR with a small extraction layer is sufficient. Database records/text can ground a hosted/local LLM; vectors are optional. Resources: pages 9–10.

13. Final Demonstration

Create/upload → extract → flag a missing/inconsistent field → show evidence → correct/replace without losing history → human decision → dashboard → cited GenAI explanation. Also show corrupt/unsupported upload, extraction failure, a blocked self-approval, an unanswerable question and a before/after or small comparison of the innovation.

RESOURCES / BOTH PROBLEMS

Datasets & Usage Notes

Dataset and resource names below are **clickable**. Choose one suitable source/subset, not every dataset. These supply model inputs and selected labels, not complete users, approvals, batch histories or business policies. Prepare those records separately.

Product Quality Inspection

[Q1] Casting product images · Kaggle Good/defective casting images; a practical binary-classification route. They do not provide separate defect-category or location labels. Check splits, augmented duplicates and reuse terms.

[Q2] MVTec AD · original publisher Normal training images; normal/anomalous test images and localisation labels. Use one category. Do not train on test defects and report that set as held out. The draft notes CC BY-NC-SA 4.0 (non-commercial).

[Q3] Severstal: Steel Defect Detection · Kaggle Steel images with four defect classes. Images may have no defect or multiple classes; preserve this when deriving a classification subset. Segmentation is optional. Check competition download/access rules.

[Q4] Manufacturing quality-inspection study Industry reading on a visual classifier used for quality inspection and the casting-data context; not a required implementation.

Document Verification

[D1] SROIE datasetv2 · Kaggle mirror English scanned receipts with OCR and selected key fields. A reorganised mirror, not invoice-approval data; receipts are not purchase orders. The listing uses an “Other” licence: check original terms, not only a code licence.

[D2] FATURA · Zenodo, version 1 Generated invoice layouts with text, field classes and boxes; the linked record describes 10,000 images from 50 templates. Use a subset and check release terms. Unseen-template tests are needed for new-layout claims.

[D3] FUNSD · original project 199 annotated scanned forms with entity/word relationships, useful for key-value extraction. The draft notes non-commercial research/education use and an 18+ age condition. No approval/authenticity labels. Terms of use

[D4] CORD · original project Indonesian receipts with OCR/structured parsing. The public release has 1,000 receipts, not the full research collection; some research fields are not public. Check language/schema. The draft notes CC BY 4.0. CORD v2 data/viewer

Prepare the Application Data Separately

Create clearly labelled fictional users, products/batches or requests, timestamps, reviews, actions and decisions. Keep original source IDs and label mappings. These document datasets do not establish approval, fraud or authenticity; synthetic factory histories do not prove defect causes.

Illustrative starter data, not a quota: around 20–30 cases per chosen problem. For quality, use one family, a few batches, normal/defective/uncertain examples and follow-up actions. For documents, include valid/incomplete/conflicting/repeated cases, a replacement and several decision states. These counts are for the demo, not model training.

Access note: Kaggle may require login/rule acceptance. Check selected-release licences, attribution and redistribution rules; code licences do not automatically cover data. An authorised alternative is acceptable. Links and dataset notes are retained from the longer draft; current access/terms were not re-verified in this edit.

SHARED GUIDANCE / BOTH PROBLEMS

Submission Checklist & Tool Resources

Shared Submission Checklist

Application & backend: source code, working UI and APIs, persistent data, permission checks, traceable status/version history, database/schema and architecture diagrams. Include API documentation, README, local setup, safe environment-variable template, demo accounts and seed-data instructions.

ML evidence: dataset/version/subset, label or field mapping, preprocessing, learned component/model version, held-out split, measured results and representative errors. Keep originals and near-duplicates together across splits; save a small reproducible test manifest. Pretrained models still need task-specific evaluation.

GenAI evidence: retrieval method and cited outputs; test a supported question, unknown case, contradictory premise and unauthorised-data request. Check citations and document versions, not just plausible wording. Keep permissions outside the model and uploaded content separate from instructions.

Demonstration & innovation: complete normal and exception cases, clear recovery behaviour and one tested team-designed improvement. Distinguish application rules, ML predictions and generated text. Identify pretrained services, borrowed code and limitations; no hard-coded predictions or fabricated metrics.

Scope: one product family or document scenario, one organisation and a few roles are sufficient. Reliable local execution is acceptable. AI coding tools may be used, but teams must explain the implementation. Optional ideas are not additional mandatory requirements.

Suggested Technology Scope & References

Choose any frontend/mobile toolkit, API backend, relational/document database, file storage and suitable learned/GenAI tools. SQL/database retrieval is enough; vectors, agents and complex cloud infrastructure are optional. Pin versions and check software/model licences separately from data.

[T1] PyTorch transfer learning — adapt an existing image classifier.

[T2] scikit-learn evaluation metrics · [T3] Data leakage & common pitfalls

[T4] Tesseract OCR · [T5] PaddleOCR — existing OCR/document-processing routes.

[T6] Sentence Transformers semantic search — learned similarity/retrieval.

[T7] React tutorial · FastAPI tutorial — optional frontend/backend references.

[T8] OWASP file-upload guidance — file types, size, storage and access controls.

[T9] OWASP prompt-injection guidance — untrusted document content and safe retrieval/actions.

Terms & Organiser Preparation

Terms: OCR = text extraction from images/scans; ML = machine learning; GenAI = generative AI; RAG = retrieving relevant stored content before generating an answer. No particular framework, model or vector database is mandated.

Before release: test a starter dataset for each problem, prepare fictional workflow data and a small held-out demo set. Document access/usage conditions; complete dataset downloads and every annotation were not audited in the original resource review.